

计算机网络实验报告（四）

实验题目	端到端网络协议分析
报告日期	2025.12.23
姓名	熊宇祺
学号	09023112

一. 实验目的

通过实验，巩固 Wireshark 网络抓包工具的使用方法，在真实网络环境下捕捉和分析端到端网络应用数据包，掌握 HTTP、DNS、TCP、UDP 等应用层及传输层端到端协议报文的结构，深入理解相关协议的特点与工作原理。

二. 实验内容

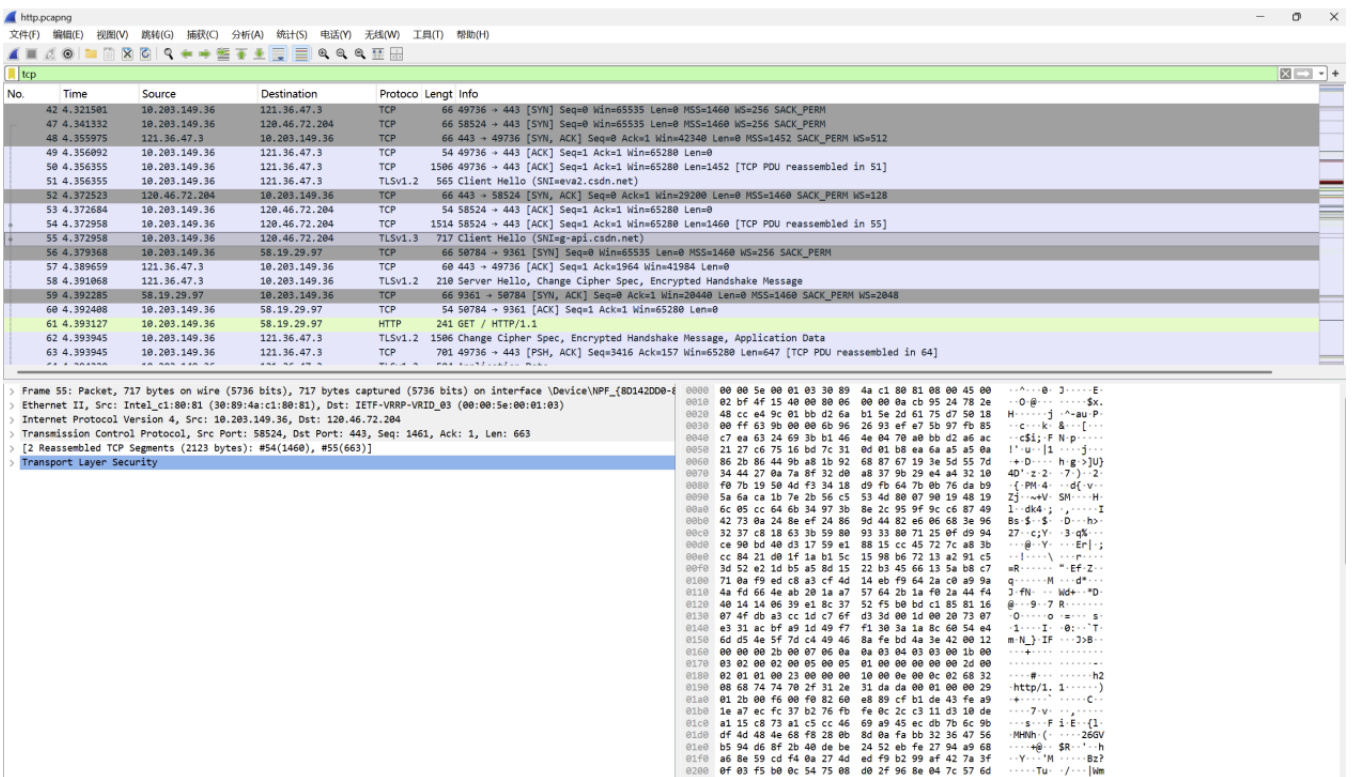
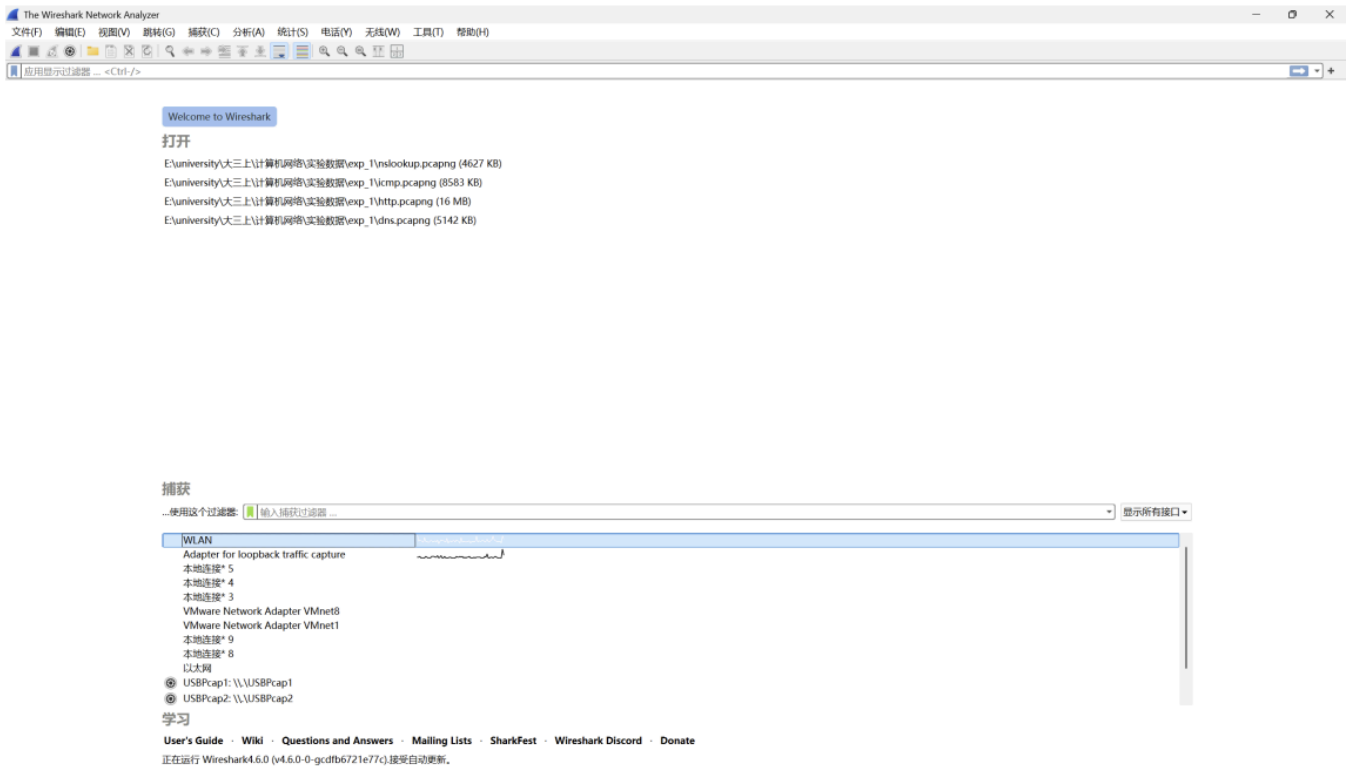
- 学习 Wireshark 网络抓包工具的基本操作，掌握**捕获过滤器**和**显示过滤器**的使用方法。
- 访问 Web 网站并捕捉数据包，分析 **HTTP 协议报文格式和交互过程**，观察 **TCP 协议报文格式**，分析 TCP 协议采用的**可靠传输机制**、以及 **TCP 连接建立和关闭的交互过程**。
- 使用 **nslookup** 命令查询域名并捕捉数据包，分析 **DNS 协议报文格式和交互过程**，同时分析 **UDP 协议报文格式和交互过程**。

三. 实验步骤（分析解释 + 系统截图）

1. Wireshark基本操作与过滤器使用

Wireshark是本次实验的核心工具。其主界面主要分为三个窗格：

- 分组列表窗格 (Packet List Pane)**: 按时间顺序显示捕获到的每个数据包的概要信息。
- 分组详情窗格 (Packet Details Pane)**: 以协议分层的树状结构显示选定数据包的详细内容。
- 分组字节流窗格 (Packet Bytes Pane)**: 以十六进制和ASCII码显示数据包的原始数据。



在实验中，**显示过滤器**起到了至关重要的作用。它允许我们在捕获海量数据后，仅关注我们感兴趣的协议或会话，语法灵活，是网络分析的利器。例如，输入 `dns` 可以只查看DNS报文，输入 `tcp.port == 80` 可以只查看访问HTTP服务端口的TCP报文。

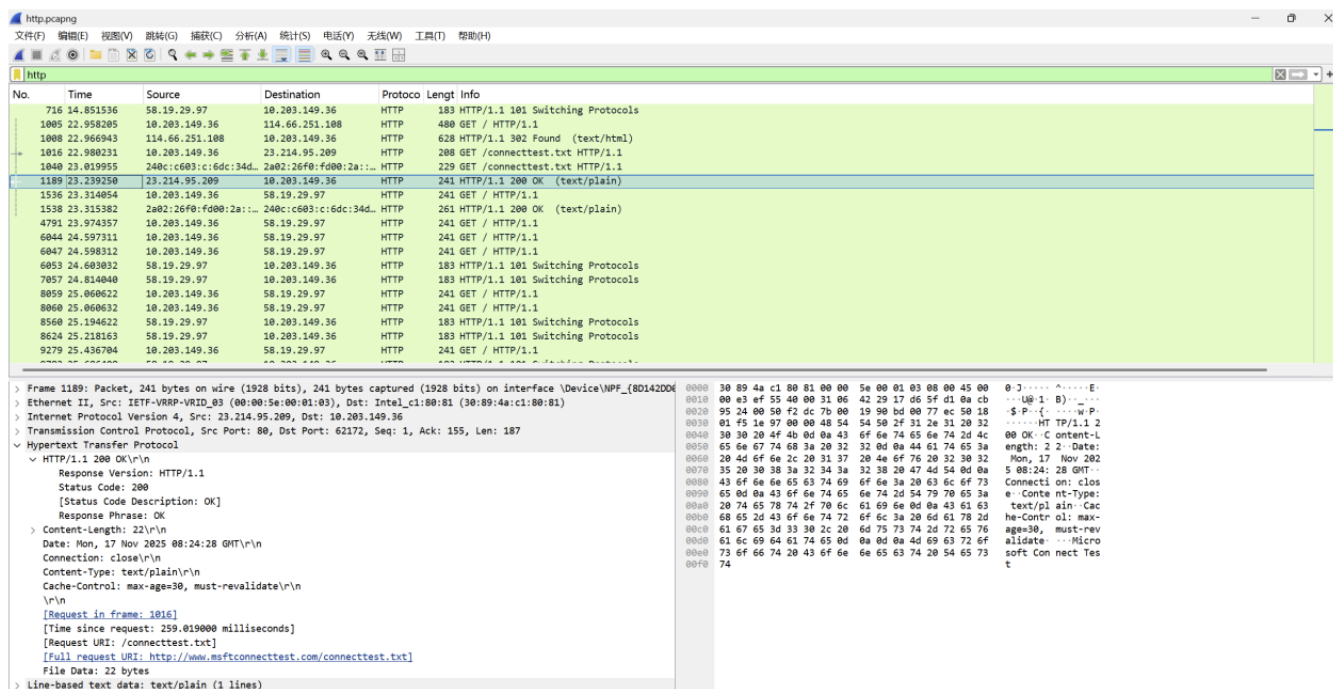
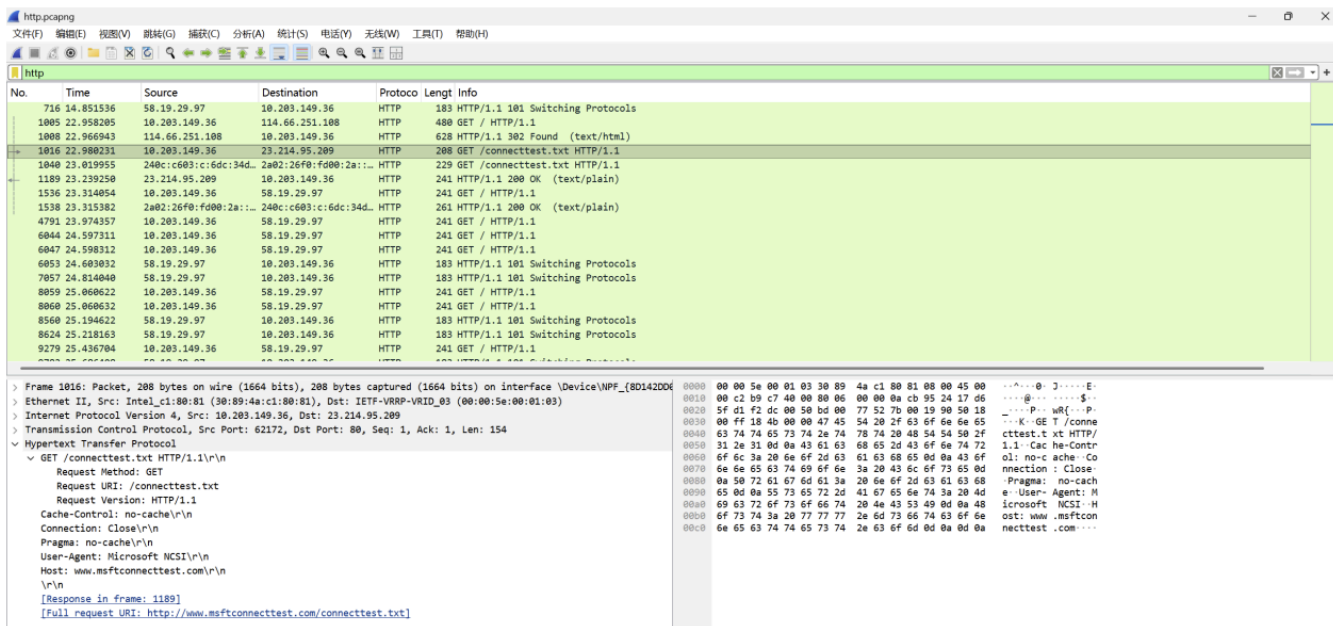
2. HTTP协议报文格式和交互过程

操作步骤：

1. **清理浏览器缓存**：在浏览器设置中或使用快捷键 `Ctrl+Shift+Delete` 清除“缓存的图片和文件”，以强制浏览器重新向服务器请求所有资源。

2. 捕获与分析：启动Wireshark，访问一个纯HTTP网站（http://info.cern.ch）。本次捕获到的是一个后台程序对 connecttest.txt 文件的请求。

实验截图：



HTTP（超文本传输协议）是应用层的核心协议之一，工作在TCP之上。

• HTTP请求报文 (Request, #1016):

◦ 请求行 (Request Line): GET /connecttest.txt HTTP/1.1

- GET: 请求方法，表示希望从服务器获取资源。
- /connecttest.txt: URI (统一资源标识符)，指明了请求资源的路径。
- HTTP/1.1: 协议版本。
- 请求头部 (Headers):
 - Host: www.msftconnecttest.com: 指明请求的目标服务器域名。

- `User-Agent:Microsoft NCSI`: 包含了发起请求的客户端（浏览器、程序）的信息。
- `Connection: Keep-Alive`: 建议服务器在完成本次响应后保持TCP连接，以便后续请求复用，减少连接建立的开销。
- **请求正文 (Body)**: 本次GET请求没有正文。
- **HTTP响应报文 (Response, #1189)**:
 - **状态行 (Status Line)**: `HTTP/1.1 200 OK`
 - `HTTP/1.1`: 协议版本。
 - `200`: **状态码**，表示请求已成功。
 - `OK`: **原因短语**，对状态码的简要描述。
 - **响应头部 (Headers)**:
 - `Content-Type: text/plain`: 指明响应正文的媒体类型为纯文本。
 - `Content-Length: 22`: 指明响应正文的长度（字节）。
 - `Date: Mon, 17 Nov 2025 08:24:28 GMT`: 响应生成的时间。
 - **响应正文 (Body)**: 在Wireshark的“Line-based text data”部分，我们可以直接看到 `connecttest.txt` 文件的内容。这清晰地证明了HTTP是**明文传输**的，任何中间节点都能轻易地读取传输内容，这也是HTTPS加密传输被广泛应用的原因。

3. TCP协议分析

3.1 定位并隔离TCP流

为了分析一个完整的TCP会话，而不是混杂在数千个无关报文中，我们使用了Wireshark的“追踪TCP流”功能。通过在一个已知的HTTP报文（#19号报文）上右键选择此功能，Wireshark自动生成过滤器 `tcp.stream eq 4`，将与该会话相关的所有TCP报文（从建立到断开）精确地隔离出来。

实验截图：



The screenshot shows the Wireshark interface with the filter `tcp.stream eq 4` applied. The selected packet is a GET request to `drive.wps.cn`. The packet details pane shows the following structure:

```
GET /update?type=switch&wpsver=12.1.0.23542&channel=00019.00000943&qingver=12.1.0.23542&language=zh_CN HTTP/1.1
Host: drive.wps.cn
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,en,*
User-Agent: Mozilla/5.0

HTTP/1.1 200 OK
Server: volcalb
Date: Mon, 17 Nov 2025 08:51:54 GMT
Content-Type: application/json;charset=utf-8
Content-Length: 168
Connection: keep-alive
x-envoy-upstream-service-time: 19
x-kso-auth-status: notLogin
x-kso-auth-raw-error: userNotLogin
X-KLB: 2
x-request-id: 682bf81fd426b5b4cb71e80e7aa1877b
Expires: Tue, 18 Nov 2025 08:51:54 GMT
Cache-Control: max-age=86400
Nginx-Cache: HIT
Vary: Accept-Encoding

{"QPush":"true","cryptcookie":"true","mobilePush":"false","qiniu":"open","result":"ok","roamHomepageSettingGuid":"true","showguide":"false","updateTipInteval":"36000"}
```

No.	Time	Source	Destination	Protocol	Length	Info
19	1.465419	10.203.149.36	180.184.103.12	TCP	66	57183 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
24	1.488050	180.184.103.12	10.203.149.36	TCP	66	80 → 57183 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1460 SACK_PERM WS=1024
25	1.488252	10.203.149.36	180.184.103.12	TCP	54	57183 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
27	1.489764	10.203.149.36	180.184.103.12	HTTP	299	GET /update?type=switch&wpsver=12.1.0.23542&channel=00019.00000943&qingver=12.1.0.23542&language=zh_CN HTTP/1.1
33	1.512679	180.184.103.12	10.203.149.36	TCP	60	80 → 57183 [ACK] Seq=1 Ack=246 Win=43008 Len=0
35	1.513494	180.184.103.12	10.203.149.36	HTTP/1.1	655	HTTP/1.1 200 OK, JSON (application/json)
66	1.560765	10.203.149.36	180.184.103.12	TCP	54	57183 → 80 [ACK] Seq=246 Ack=602 Win=64768 Len=0
1607	76.516578	180.184.103.12	10.203.149.36	TCP	60	80 → 57183 [FIN, ACK] Seq=602 Ack=246 Win=43008 Len=0
1608	76.516697	10.203.149.36	180.184.103.12	TCP	54	57183 → 80 [ACK] Seq=246 Ack=603 Win=64768 Len=0
1609	76.517094	10.203.149.36	180.184.103.12	TCP	54	57183 → 80 [FIN, ACK] Seq=246 Ack=603 Win=64768 Len=0
1612	76.547093	180.184.103.12	10.203.149.36	TCP	60	80 → 57183 [ACK] Seq=603 Ack=247 Win=43008 Len=0

3.2 TCP连接建立过程（三次握手机制）

在隔离出的TCP流的顶部，我们清晰地看到了连接建立的过程：

- **第一次握手 (#19):**
 - **方向:** Client (10.203.149.36) -> Server (180.184.103.12)
 - **报文分析:** 这是一个 [SYN] 报文，标志位中的 SYN 被置为1。客户端以此发起连接请求，并发送自己的初始序列号 Seq=0。此时，客户端TCP状态从 CLOSED 迁移到 SYN_SENT。
- **第二次握手 (#24):**
 - **方向:** Server -> Client
 - **报文分析:** 这是一个 [SYN, ACK] 报文，标志位中的 SYN 和 ACK 都被置为1。服务器用 SYN 表示同意建立连接并发送自己的初始序列号 Seq=0；用 ACK (Ack=1，即客户端Seq+1) 表示已成功收到客户端的请求。此时，服务器TCP状态从 LISTEN 迁移到 SYN_RCVD。
- **第三次握手 (#25):**
 - **方向:** Client -> Server
 - **报文分析:** 这是一个 [ACK] 报文。客户端以此确认收到了服务器的 SYN (Ack=1，即服务器Seq+1)。此报文发送后，客户端TCP状态迁移到 ESTABLISHED；服务器收到此报文后，也迁移到 ESTABLISHED 状态。至此，可靠的双向连接建立完成。

3.3 TCP连接关闭过程（四次挥手机制）

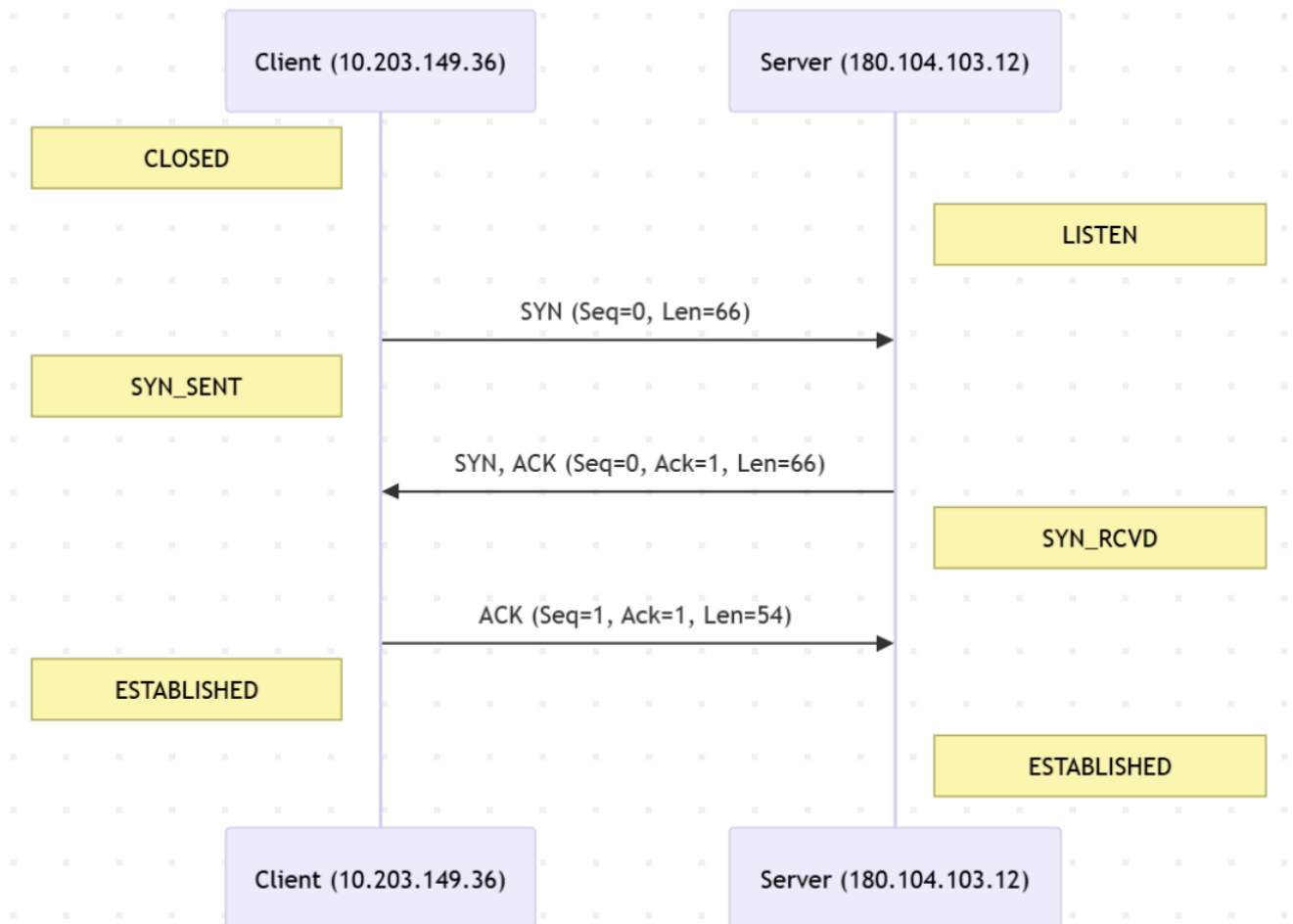
在TCP流的底部，我们观察到了连接的释放过程（本次由服务器主动发起）：

- **第一次挥手 (#1607):**
 - **方向:** Server -> Client
 - **报文分析:** 服务器发送一个 [FIN, ACK] 报文，FIN 位置1，表示服务器已没有数据要发送，请求关闭从服务器到客户端方向的连接。服务器进入 FIN_WAIT_1 状态。
- **第二次挥手 (#1608):**
 - **方向:** Client -> Server
 - **报文分析:** 客户端收到 FIN 后，发送一个 [ACK] 报文作为确认。此时，从服务器到客户端的连接关闭，但从客户端到服务器的连接仍然可以发送数据。客户端进入 CLOSE_WAIT 状态。
- **第三次挥手 (#1609):**
 - **方向:** Client -> Server
 - **报文分析:** 客户端在确认自己也没有数据要发送后，也发送一个 [FIN, ACK] 报文，请求关闭从客户端到服务器方向的连接。客户端进入 LAST_ACK 状态。

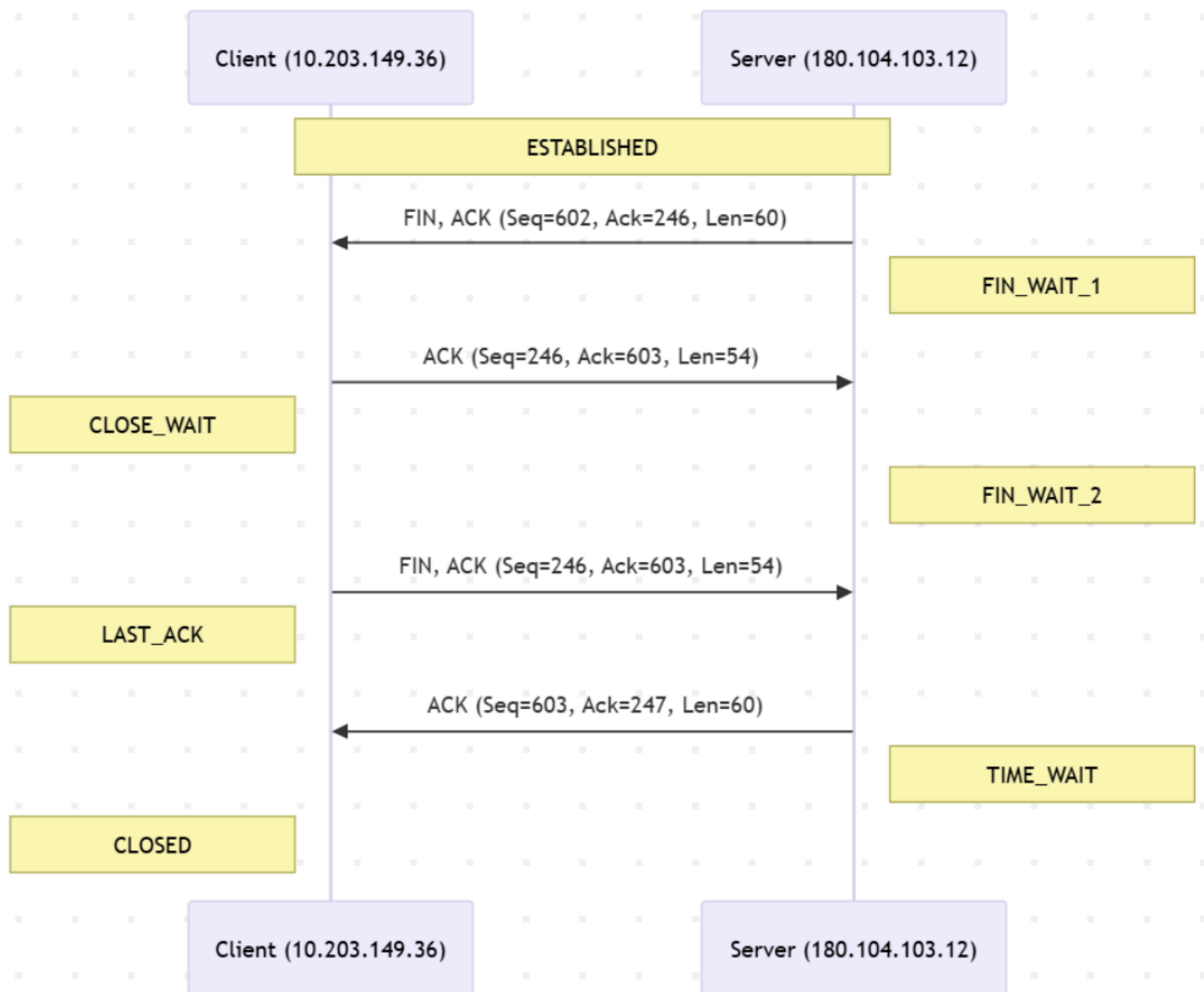
- 第四次挥手 (#1612):
 - 方向: Server -> Client
 - 报文分析: 服务器收到客户端的 FIN 后, 发送最后一个 [ACK] 报文进行确认。服务器随即进入 TIME_WAIT 状态 (等待2MSL以确保报文可靠送达), 之后进入 CLOSED。客户端收到这个 ACK 后, 立即进入 CLOSED 状态。连接彻底释放。

3.4 TCP连接时序图与状态变迁

TCP三次握手时序图



TCP四次挥手时序图 (服务器主动关闭)



注意：这里由于mermaid语法问题，无法画出斜线，实际每一个箭头的时戳要晚于对应的箭尾的时戳

3.5 TCP协议报文格式

我们选择三次握手中的第一个报文（#19 [SYN] 包）进行详细的字段级分析。

实验截图：

Transmission Control Protocol, Src Port: 57183, Dst Port: 80, Seq: 0, Len: 0

Source Port: 57183

Destination Port: 80

[Stream index: 4]

[Stream Packet Number: 1]

▼ [Conversation completeness: Complete, WITH_DATA (31)]

..0. = RST: Absent

...1 = FIN: Present

.... 1... = Data: Present

.... .1.. = ACK: Present

.... ..1. = SYN-ACK: Present

.... ...1 = SYN: Present

[Completeness Flags: ·FDASS]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 781500524

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 0

Acknowledgment number (raw): 0

1000 = Header Length: 32 bytes (8)

> Flags: 0x002 (SYN)

Window: 65535

[Calculated window size: 65535]

Checksum: 0xbdda [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

▼ Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted

▼ TCP Option - Maximum segment size: 1460 bytes

Kind: Maximum Segment Size (2)

Length: 4

MSS Value: 1460

> TCP Option - No-Operation (NOP)

▼ TCP Option - Window scale: 8 (multiply by 256)

Kind: Window Scale (3)

Length: 3

Shift count: 8

[Multiplier: 256]

> TCP Option - No-Operation (NOP)

> TCP Option - No-Operation (NOP)

▼ TCP Option - SACK permitted

Kind: SACK Permitted (4)

Length: 2

▼ [Timestamps]

[Time since first frame in this TCP stream: 0.00000000 seconds]

[Time since previous frame in this TCP stream: 0.00000000 seconds]

[Client Contiguous Streams: 1]

[Server Contiguous Streams: 1]

TCP首部字段详细分析表

字段 (Field)	捕获值 (Value)	定义与含义分析
Source Port	57183	源端口。 客户端操作系统为本次连接动态分配的端口号，用于标识本机上的通信进程。
Destination Port	80	目的端口。 HTTP服务的熟知端口号，指明了服务器上提供服务的应用程序。
Sequence Number	0 (Relative)	序列号。 本次连接的初始序列号(ISN)，标志着客户端发送数据流的起始位置。后续报文的序列号将在此基础上递增。
Acknowledgment Number	0	确认号。 由于这是连接发起的第一个报文，ACK标志位为0，此字段无效。
Header Length	32 bytes	头部长度的。 以4字节为单位，值为8 (8*4=32) 。因包含12字节的选项，故长度大于标准的20字节。

字段 (Field)	捕获值 (Value)	定义与含义分析
Flags	0x002 (SYN)	标志位。 该字段的 SYN 位被置为1，明确表明这是一个 请求建立连接 的同步报文。
Window size	65535	窗口大小。 用于TCP流量控制。客户端以此告知服务器，其接收缓冲区当前最多能接收65535字节的数据。
Checksum	0xbbda	校验和。 用于检验TCP头部和数据在传输中是否出错，保障数据完整性。Wireshark 通常是在操作系统内核将数据包交给网卡之前捕获到这个数据包的，里面的校验和字段还是一个无效的占位符。所以Wireshark将其标识为 未验证
Options	(12 bytes)	选项。 用于在连接建立时协商一些高级特性：
* - MSS	1460 bytes	最大报文段长度。 客户端声明其能接收的最大TCP数据段为1460字节，用于避免IP层分片。
* - SACK Permitted	1	选择性确认许可。 表明客户端支持SACK机制，可在丢包时只重传丢失的报文段，提高效率。
* - Window Scale	256	窗口扩大因子。 允许将窗口大小字段的值乘以256，以支持在高速网络下更大的接收窗口。

3.6 TCP协议的可靠传输机制

TCP（Transmission Control Protocol，传输控制协议）是一种面向连接、面向字节流的传输层协议。为了在不可靠的 IP 网络之上提供可靠的数据传输服务，TCP 设计并采用了多种可靠传输机制，主要包括以下几个方面：

- 1. 序号与确认应答机制（Sequence Number & ACK）

TCP 将应用层传下来的数据视为连续的字节流，并为每一个字节分配一个序号（Sequence Number）。

接收方在成功接收数据后，会向发送方返回确认报文（ACK），其中确认号表示“期望接收到的下一个字节序号”。

通过序号与确认号机制，TCP 可以：

- 判断数据是否按序到达；
- 发现数据是否丢失或重复；
- 保证数据的有序交付。

- 2. 超时重传机制（Timeout Retransmission）

TCP 通过设置重传定时器（Retransmission Timer）来检测数据是否丢失。

如果发送方在规定时间内未收到对应数据段的确认报文（ACK），则认为该数据段可能在传输过程中丢失，发送方会对该数据段进行重传。

该机制能够有效应对网络中的：

- 分组丢失；
- 分组延迟过大；

- 确认报文丢失等情况
- 3. **差错检测机制（校验和）**

TCP 在报文段首部中包含校验和字段（Checksum），用于检测数据在传输过程中是否发生比特错误。发送方在发送数据前计算校验和，接收方在接收数据后重新计算并进行比对。若校验失败，接收方会丢弃该报文段，不发送确认，从而触发发送方的重传机制，保证数据的正确性。
- 4. **流量控制机制（Flow Control）**

TCP 使用滑动窗口机制（Sliding Window）实现端到端的流量控制。

接收方通过通告窗口大小（Receive Window, rwnd）来告知发送方当前自身的接收能力，防止发送方发送数据过快而导致接收方缓冲区溢出。

流量控制保证了：

 - 发送速率与接收能力相匹配；
 - 数据传输过程的稳定性。
- 5. **拥塞控制机制**

虽然拥塞控制的主要目的是防止网络拥塞，但它也间接提高了 TCP 传输的可靠性。

TCP 通过慢开始、拥塞避免、快重传和快恢复等算法，动态调整发送速率，减少因网络拥塞导致的数据丢失。

4. DNS协议报文格式和交互过程

操作步骤：

1. **清空本地DNS缓存：**为确保能够捕获到完整的DNS查询过程，以管理员身份打开 cmd，执行 `ipconfig /flushdns` 命令。

```
(base) PS C:\WINDOWS\system32> ipconfig /flushdns

windows IP Configuration

Successfully flushed the DNS Resolver Cache.
```

2. **捕获与筛选：**启动Wireshark抓包，在浏览器中访问 `www.baidu.com`。停止抓包后，为精确定位，在显示过滤器中输入 `dns.qry.name == "www.baidu.com"`。

实验截图：

#354号报文记录

dns.pcapng

文件(F) 编辑(E) 视图(V) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

dns

No.	Time	Source	Destination	Protocol	Length	Info
42	8.946949	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc052 AAAA osfsr.lenovom.com
44	8.950341	172.19.2.1	10.203.149.36	DNS	171	Standard query response 0xc052 AAAA osfsr.lenovom.com CNAME fsr.cn.lenovom.com SOA ns1.dnsv5.com
352	17.545099	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc010 AAAA edge.microsoft.com
353	17.545411	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc010 AAAA edge.microsoft.com
354	17.545632	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc577 A edge.microsoft.com
359	17.548677	172.19.2.1	10.203.149.36	DNS	132	Standard query response 0xc010 AAAA edge.microsoft.com CNAME edge-microsoft-com.ax-0002.ax-msedge.net
360	17.549963	172.19.2.1	10.203.149.36	DNS	202	Standard query response 0xc010 AAAA edge.microsoft.com CNAME edge-microsoft-com.ax-0002.ax-msedge.net CNAME ax-0002.ax-msedge.net AAAA 2620:1ec:33::11 A 150.171.28.11 A 150.171.28.11
361	17.549963	172.19.2.1	10.203.149.36	DNS	178	Standard query response 0xc577 A edge.microsoft.com CNAME edge-microsoft-com.ax-0002.ax-msedge.net CNAME ax-0002.ax-msedge.net A 150.171.28.11 A 150.171.28.11
421	18.428517	10.203.149.36	172.19.2.1	DNS	86	Standard query 0xc010 AAAA browser.events.data.msn.cn
422	18.428830	10.203.149.36	172.19.2.1	DNS	86	Standard query 0xc010 AAAA browser.events.data.msn.cn
423	18.429024	10.203.149.36	172.19.2.1	DNS	86	Standard query 0xc2f5 A browser.events.data.msn.cn
445	18.434955	172.19.2.1	10.203.149.36	DNS	200	Standard query response 0xc010 AAAA browser.events.data.msn.cn CNAME global.asimov.events.data.trafficmanager.net CNAME onedscolprdeus17.westus.cloudapp.azure.com
446	18.436597	172.19.2.1	10.203.149.36	DNS	271	Standard query response 0xc010 AAAA browser.events.data.msn.cn CNAME global.asimov.events.data.trafficmanager.net CNAME onedscolprdeus20.eastus.cloudapp.azure.com
447	18.436597	172.19.2.1	10.203.149.36	DNS	223	Standard query response 0xc2f5 A browser.events.data.msn.cn CNAME global.asimov.events.data.trafficmanager.net CNAME onedscolprdeus03.australiaeast.cloudapp.azure.com
516	18.985183	10.203.149.36	172.19.2.1	DNS	79	Standard query 0xc302 HTTPS defn.nelreports.net
517	18.985465	10.203.149.36	172.19.2.1	DNS	79	Standard query 0xc302 HTTPS defn.nelreports.net
518	18.985683	10.203.149.36	172.19.2.1	DNS	79	Standard query 0xc5c1 A defn.nelreports.net
519	18.988917	10.203.149.36	172.19.2.1	DNS	211	Standard query response 0xc302 HTTPS defn.nelreports.net CNAME defn.nelreports.net.akamaiized.net CNAME a1858.dscd.akamai.net
520	18.989258	172.19.2.1	10.203.149.36	DNS	155	Standard query response 0xc302 HTTPS defn.nelreports.net CNAME defn.nelreports.net.akamaiized.net CNAME a1858.dscd.akamai.net
521	18.989258	172.19.2.1	10.203.149.36	DNS	187	Standard query response 0xc5c1 A defn.nelreports.net CNAME defn.nelreports.net.akamaiized.net CNAME a1858.dscd.akamai.net A 2.20.12.85 A 2.20.12.102
524	19.079145	10.203.149.36	172.19.2.1	DNS	71	Standard query 0xc010 AAAA cn.bing.com
525	19.080019	10.203.149.36	172.19.2.1	DNS	71	Standard query 0xc010 AAAA cn.bing.com
526	19.080574	10.203.149.36	172.19.2.1	DNS	71	Standard query 0xc010 AAAA cn.bing.com
527	19.081178	172.19.2.1	10.203.149.36	DNS	176	Standard query response 0xc010 AAAA cn.bing.com CNAME cn-bing-com.cn.a-0001.a-msedge.net SOA ns1.a-msedge.net

> Frame 354: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface \Device\NPF_{801420D0-861E-408D-B050-000000000000} (00:00:00:00:00:00:00:00)

> Ethernet II, Src: Intel E100-B1 (30:05:4c:00:00:01), Dst: IEEE VRRP-VRID_03 (00:00:5e:00:01:03)

> Internet Protocol Version 4, Src: 10.203.149.36, Dst: 172.19.2.1

> User Datagram Protocol, Src Port: 51368, Dst Port: 53

> Domain Name System (query)

Transaction ID: 0xc577

Flags: 0x0100 Standard query

Questions: 1

Answer RRs: 0

Authority RRs: 0

Additional RRs: 0

Queries

edge.microsoft.com: type A, class IN

Name: edge.microsoft.com

[Name Length: 18]

[Label Count: 3]

Type: A (1) (Host Address)

Class: IN (0x0001)

[Response In: 361]

#360号报文记录

dns.pcapng

文件(F) 编辑(E) 视图(V) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

dns

No.	Time	Source	Destination	Protocol	Length	Info
42	8.946949	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc052 AAAA osfsr.lenovom.com
44	8.950341	172.19.2.1	10.203.149.36	DNS	171	Standard query response 0xc052 AAAA osfsr.lenovom.com CNAME fsr.cn.lenovom.com SOA ns1.dnsv5.com
352	17.545099	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc010 AAAA edge.microsoft.com
353	17.545411	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc010 AAAA edge.microsoft.com
354	17.545632	10.203.149.36	172.19.2.1	DNS	78	Standard query 0xc577 A edge.microsoft.com
359	17.548677	172.19.2.1	10.203.149.36	DNS	132	Standard query response 0xc010 AAAA edge.microsoft.com CNAME edge-microsoft-com.ax-0002.ax-msedge.net
360	17.549963	172.19.2.1	10.203.149.36	DNS	202	Standard query response 0xc010 AAAA edge.microsoft.com CNAME edge-microsoft-com.ax-0002.ax-msedge.net CNAME ax-0002.ax-msedge.net AAAA 2620:1ec:33::11 A 150.171.28.11 A 150.171.28.11
361	17.549963	172.19.2.1	10.203.149.36	DNS	178	Standard query response 0xc577 A edge.microsoft.com CNAME edge-microsoft-com.ax-0002.ax-msedge.net CNAME ax-0002.ax-msedge.net A 150.171.28.11 A 150.171.28.11
421	18.428517	10.203.149.36	172.19.2.1	DNS	86	Standard query 0xc010 AAAA browser.events.data.msn.cn
422	18.428830	10.203.149.36	172.19.2.1	DNS	86	Standard query 0xc010 AAAA browser.events.data.msn.cn
423	18.429024	10.203.149.36	172.19.2.1	DNS	86	Standard query 0xc2f5 A browser.events.data.msn.cn
445	18.434955	172.19.2.1	10.203.149.36	DNS	200	Standard query response 0xc010 AAAA browser.events.data.msn.cn CNAME global.asimov.events.data.trafficmanager.net CNAME onedscolprdeus17.westus.cloudapp.azure.com
446	18.436597	172.19.2.1	10.203.149.36	DNS	271	Standard query response 0xc010 AAAA browser.events.data.msn.cn CNAME global.asimov.events.data.trafficmanager.net CNAME onedscolprdeus20.eastus.cloudapp.azure.com
447	18.436597	172.19.2.1	10.203.149.36	DNS	223	Standard query response 0xc2f5 A browser.events.data.msn.cn CNAME global.asimov.events.data.trafficmanager.net CNAME onedscolprdeus03.australiaeast.cloudapp.azure.com
516	18.985183	10.203.149.36	172.19.2.1	DNS	79	Standard query 0xc302 HTTPS defn.nelreports.net
517	18.985465	10.203.149.36	172.19.2.1	DNS	79	Standard query 0xc302 HTTPS defn.nelreports.net
518	18.985683	10.203.149.36	172.19.2.1	DNS	79	Standard query 0xc5c1 A defn.nelreports.net
519	18.988917	10.203.149.36	172.19.2.1	DNS	211	Standard query response 0xc302 HTTPS defn.nelreports.net CNAME defn.nelreports.net.akamaiized.net CNAME a1858.dscd.akamai.net
520	18.989258	172.19.2.1	10.203.149.36	DNS	155	Standard query response 0xc302 HTTPS defn.nelreports.net CNAME defn.nelreports.net.akamaiized.net CNAME a1858.dscd.akamai.net
521	18.989258	172.19.2.1	10.203.149.36	DNS	187	Standard query response 0xc5c1 A defn.nelreports.net CNAME defn.nelreports.net.akamaiized.net CNAME a1858.dscd.akamai.net A 2.20.12.85 A 2.20.12.102
524	19.079145	10.203.149.36	172.19.2.1	DNS	71	Standard query 0xc010 AAAA cn.bing.com
525	19.080019	10.203.149.36	172.19.2.1	DNS	71	Standard query 0xc010 AAAA cn.bing.com
526	19.080574	10.203.149.36	172.19.2.1	DNS	71	Standard query 0xc010 AAAA cn.bing.com
527	19.081178	172.19.2.1	10.203.149.36	DNS	176	Standard query response 0xc010 AAAA cn.bing.com CNAME cn-bing-com.cn.a-0001.a-msedge.net SOA ns1.a-msedge.net

> User Datagram Protocol, Src Port: 53, Dst Port: 63413

> Domain Name System (response)

Transaction ID: 0xc010

Flags: 0x8100 Standard query response, No error

Questions: 1

Answer RRs: 4

Authority RRs: 0

Additional RRs: 0

Queries

edge.microsoft.com: type AAAA, class IN

Name: edge.microsoft.com

[Name Length: 18]

[Label Count: 3]

Type: AAAA (28) (IP6 Address)

Class: IN (0x0001)

Answers

edge.microsoft.com: type CNAME, class IN, cname edge-microsoft-com.ax-0002.ax-msedge.net

edge-microsoft-com.ax-0002.ax-msedge.net: type CNAME, class IN, cname ax-0002.ax-msedge.net

ax-0002.ax-msedge.net: type AAAA, class IN, addr 2620:1ec:33::11

ax-0002.ax-msedge.net: type AAAA, class IN, addr 2620:1ec:33::11

[Request In: 353]

[Time: 4.552000 milliseconds]

结果详细分析：

DNS协议负责将人类易于记忆的域名转换为机器能够识别的IP地址。从捕获结果看，一次域名解析通常包含对A记录（IPv4）和AAAA记录（IPv6）的并行查询。

以A记录查询为例进行深度分析（#354，#360号报文）：

- DNS查询报文（#354为例）：
 - 客户端 10.203.149.36 向本地DNS服务器 172.19.2.1 发送了一个查询请求。根据分组列表窗格，#354号报文的Info列显示为 Standard query 0xc010 AAAA edge.microsoft.com。

- **核心字段分析 (参照类似查询报文的结构):**

- **Transaction ID:** 0xc010。这是一个由客户端生成的16位随机数，作为本次“对话”的唯一标识。
- **Flags:** 标记为Standard query，表明这是一个由客户端发起的标准查询请求。
- **Questions:** 数量为1，表明该DNS报文中只包含一个问题。
- **Queries (问题部分):**
 - Name: edge.microsoft.com: 指明需要查询的域名。
 - Type: AAAA (IPv6 Address): 指明查询的记录类型为AAAA记录，即目标主机的IPv6地址。

- **DNS响应报文 (#360为例):**

- DNS服务器 172.19.2.1 向客户端 10.203.149.36 返回了针对edge.microsoft.com的AAAA记录查询结果。这张截图的“分组详情”窗格为我们提供了极其丰富的分析信息。

- **核心字段详细剖析:**

- **Transaction ID:** 0xc010。与#354号查询报文的ID完全一致，客户端据此确认这是对它之前提问的回答。
- **Flags:** 0x8180 (Standard query response, No error)。这是一个16位的标志字段：
 - 第一位为1 (8...)，表示这是一个**响应 (Response)** 报文。
 - Recursion Desired位为1，Recursion Available位为1 (...1...)，表明服务器支持并完成了递归查询。
 - RCODE为0 (...0 No error)，表示查询成功，没有任何错误。
- **Counts (计数器):**
 - Questions: 1: 响应中重复了一遍客户端提出的1个问题。
 - Answer RRs: 3: **回答资源记录(Resource Records)**的数量为3。这是本次响应的核心。
 - Authority RRs: 0 和 Additional RRs: 0: 授权和附加记录数为0。
- **Answers (回答部分):** 包含了3条资源记录，这揭示了一个**CNAME链**的解析过程：
 1. **第一条回答 (CNAME记录):**
 - Name: edge.microsoft.com
 - Type: CNAME (Canonical Name)
 - CNAME: edge-microsoft-com.ax-0002.ax-msedge.net
 - **分析:** 它告诉客户端，你查询的edge.microsoft.com只是一个别名，它的“真名”是edge-microsoft-com.ax-0002.ax-msedge.net。这是内容分发网络 (CDN) 的典型应用，通过别名将用户引导到CDN的边缘节点域名。
 2. **第二条回答 (CNAME记录):**
 - Name: edge-microsoft-com.ax-0002.ax-msedge.net
 - Type: CNAME (Canonical Name)
 - CNAME: ax-0002.ax-msedge.net
 - **分析:** 进一步告诉客户端，上一个“真名”还是一个别名，最终的“真名”是ax-0002.ax-msedge.net。客户端必须沿着这个链条继续解析。
 3. **第三条回答 (AAAA记录):**
 - Name: ax-0002.ax-msedge.net

- Type: AAAA (IPv6 Address)
- Address: 2620:1ec:33::11
- **分析:** 这是最后一条回答，也是最终的回答。它提供了最终“真名”所对应的IPv6地址。在完成这一系列的CNAME跳转后，客户端终于获得了可用于建立网络连接的目标IP地址。这个过程完美地展示了现代大型网站如何通过多层别名来灵活地管理和分发流量。

5. nslookup 查询域名并捕捉数据包

操作步骤:

在Wireshark中设置 dns 过滤器并抓包，同时在 cmd 中执行 `nslookup www.baidu.com`。

实验截图:

```
(base) PS C:\WINDOWS\system32> nslookup www.baidu.com
```

Server: Unknown

Address: 172.19.2.1

Non-authoritative answer:

Name: www.a.shifen.com

Addresses: 2408:873d:22:18cb:0:ff:b037:e6d8

2408:873d:22:1a91:0:ff:b013:cf81

153.3.238.127

153.3.238.28

Aliases: www.baidu.com

No.	Time	Source	Destination	Protocol	Length	Info
67	4.212523	10.203.149.36	172.19.2.1	DNS	83	Standard query 0x0001 PTR 1.2.19.172.in-addr.arpa
68	4.216381	172.19.2.1	10.203.149.36	DNS	118	Standard query response 0x0001 No such name PTR 1.2.19.172.in-addr.arpa SOA 19.172.in-addr.arpa
69	4.219450	10.203.149.36	172.19.2.1	DNS	73	Standard query 0x0002 A www.baidu.com
70	4.221446	172.19.2.1	10.203.149.36	DNS	132	Standard query response 0x0002 A www.baidu.com CNAME www.a.shifen.com A 153.3.238.127 A 153.3.238.28
71	4.229286	10.203.149.36	172.19.2.1	DNS	73	Standard query 0x0003 AAAA www.baidu.com
72	4.231811	172.19.2.1	10.203.149.36	DNS	156	Standard query response 0x0003 AAAA www.baidu.com CNAME www.a.shifen.com AAAA 2408:873d:22:18cb:0:ff:b037:e6d8 AAAA 2408:873d:22:1a91:0:ff:b013:cf81
76	4.862701	10.203.149.36	172.19.2.1	DNS	76	Standard query 0x0008 A co02.jurassic.net
77	4.863042	10.203.149.36	172.19.2.1	DNS	76	Standard query 0x00c9 AAAA co02.jurassic.net
78	4.864090	172.19.2.1	10.203.149.36	DNS	549	Standard query response 0x00c9 A co02.jurassic.net A 184.233.220.45 NS h.gtld-servers.net NS g.gtld-servers.net NS e.gtld-servers.net NS
79	4.866006	172.19.2.1	10.203.149.36	DNS	76	Standard query 0x00c9 AAAA co02.jurassic.net
436	21.773262	10.203.149.36	172.19.2.1	DNS	72	Standard query 0xbdeb A hq.sina.cn
437	21.774839	172.19.2.1	10.203.149.36	DNS	161	Standard query response 0xbdeb A hq.sina.cn CNAME idc-hq-th.sina.cn CNAME idc-hq-bx.sina.cn CNAME idc-hq-bx6.sina.cn A 123.125.107.29
929	37.407012	10.203.149.36	172.19.2.1	DNS	73	Standard query 0xc50 A www.baidu.com
930	37.407327	10.203.149.36	172.19.2.1	DNS	73	Standard query 0xb6fb AAAA www.baidu.com
931	37.413626	172.19.2.1	10.203.149.36	DNS	132	Standard query response 0xc50 A www.baidu.com CNAME www.a.shifen.com A 153.3.238.127 A 153.3.238.28
932	37.429792	172.19.2.1	10.203.149.36	DNS	156	Standard query response 0xb6fb AAAA www.baidu.com CNAME www.a.shifen.com AAAA 2408:873d:22:18cb:0:ff:b037:e6d8 AAAA 2408:873d:22:1a91:0:ff:b013:cf81
2505	56.053930	10.203.149.36	172.19.2.1	DNS	76	Standard query 0xb67d AAAA wn.pos.baidu.com
2506	56.060109	172.19.2.1	10.203.149.36	DNS	134	Standard query response 0xb67d AAAA wn.pos.baidu.com CNAME wn.pos.e.shifen.com AAAA 2408:873d:1022:1b77:0:ff:b0d5:c11e
5440	97.557148	10.203.149.36	172.19.2.1	DNS	73	Standard query 0x8ca8 A www.baidu.com
5441	97.558292	10.203.149.36	172.19.2.1	DNS	73	Standard query 0xf11e AAAA www.baidu.com
5442	97.564727	172.19.2.1	10.203.149.36	DNS	156	Standard query response 0xf11e AAAA www.baidu.com CNAME www.a.shifen.com AAAA 2408:873d:22:1a91:0:ff:b013:cf81 AAAA 2408:873d:22:18cb:0:ff:b037:e6d8
5443	97.564727	172.19.2.1	10.203.149.36	DNS	132	Standard query response 0x8ca8 A www.baidu.com CNAME www.a.shifen.com A 153.3.238.127 A 153.3.238.28
5499	107.952082	10.203.149.36	172.19.2.1	DNS	88	Standard query 0x000a AAAA galileo.telemetry.tencent.com
5500	107.960246	172.19.2.1	10.203.149.36	DNS	164	Standard query response 0x000a AAAA galileo.telemetry.tencent.com CNAME ins-49sdgxp.ias.tencent-cloud.net AAAA 2409:8c1e:75b0:1121:11a
5701	119.689746	10.203.149.36	172.19.2.1	DNS	72	Standard query 0xe9f0 A hq.sina.cn
5702	119.690533	10.203.149.36	172.19.2.1	DNS	75	Standard query 0xe9f0 A hq.sina.cn
5703	119.691582	172.19.2.1	10.203.149.36	DNS	139	Standard query response 0xe9f0 A hq.sina.cn CNAME idc-hq-xxg.sina.cn CNAME idc-hq-xxg6.sina.cn A 111.13.134.169
5705	119.693104	172.19.2.1	10.203.149.36	DNS	243	Standard query response 0xa297 AAAA image.sina.cn CNAME images.sina.gslb.sinaedge.com CNAME acksmall.grid.sinaedge.com CNAME wml.sinaimg.cn.w.alikunlun.co

分析:

cmd 的输出结果与Wireshark捕获的数据包形成了完美的对应。Wireshark的捕获揭示了 nslookup 命令在后台的真实动作：它是标准DNS协议的一个应用客户端。它发出了与浏览器解析时类似的A和AAAA记录查询，并且收到的响应（包括CNAME和IP地址）与 cmd 窗口中打印的结果完全一致。这深刻地揭示了上层工具与底层协议之间的实现关系。

查询IPv4地址

- **发出请求 (#69号报文):**
 - **动作:** 您的电脑 (10.203.149.36) 向本地 DNS 服务器 (172.19.2.1) 发送了一个标准的 DNS 查询。
 - **内容:** “请问 www.baidu.com 的 A 记录 (IPv4 地址) 是什么?”
- **收到响应 (#70号报文):**

- Domain Name System (response)
 - Transaction ID: 0x0002
 - > Flags: 0x8180 Standard query response, No error
 - Questions: 1
 - Answer RRs: 3
 - Authority RRs: 0
 - Additional RRs: 0
 - ✓ Queries
 - ✓ www.baidu.com: type A, class IN
 - Name: www.baidu.com
 - [Name Length: 13]
 - [Label Count: 3]
 - Type: A (1) (Host Address)
 - Class: IN (0x0001)
 - ✓ Answers
 - ✓ www.baidu.com: type CNAME, class IN, cname www.a.shifen.com
 - Name: www.baidu.com
 - Type: CNAME (5) (Canonical NAME for an alias)
 - Class: IN (0x0001)
 - Time to live: 854 (14 minutes, 14 seconds)
 - Data length: 15
 - CNAME: www.a.shifen.com
 - ✓ www.a.shifen.com: type A, class IN, addr 153.3.238.127
 - Name: www.a.shifen.com
 - Type: A (1) (Host Address)
 - Class: IN (0x0001)
 - Time to live: 0 (0 seconds)
 - Data length: 4
 - Address: 153.3.238.127
 - ✓ www.a.shifen.com: type A, class IN, addr 153.3.238.28
 - Name: www.a.shifen.com
 - Type: A (1) (Host Address)
 - Class: IN (0x0001)
 - Time to live: 0 (0 seconds)
 - Data length: 4
 - Address: 153.3.238.28
- [Request In: 69]
- [Time: 1.996000 milliseconds]

- **动作:** DNS 服务器返回了一个信息量丰富的响应。这个响应分为两部分，这与 nslookup 在屏幕上打印出的内容完全一致。
- **第一部分 (别名):** 服务器首先回答：“www.baidu.com 只是一个别名 (CNAME)，它的真实规范名称是 www.a.shifen.com。”
- **第二部分 (真实地址):** 紧接着，为了方便客户端，服务器直接提供了这个真实名称的 A 记录：“www.a.shifen.com 的 IPv4 地址是 153.3.238.127 和 153.3.238.28。”
- **联动:** nslookup 会在屏幕上先打印出 CNAME 信息，再打印出最终的 IP 地址，源头就在这里。

查询IPv6地址

现代操作系统和应用为了支持 IPv6，通常会同时发起 A 和 AAAA 记录的查询。

- **发出请求 (#71号报文):**
 - **动作:** 电脑几乎在同一时间，又向 DNS 服务器发送了另一个查询。
 - **内容理解:** “请问 www.baidu.com 的 AAAA 记录 (IPv6 地址) 是什么？”
- **收到响应 (#72号报文):**
- **动作:** DNS 服务器返回了与查询 A 记录时结构类似的响应。
- **第一部分 (别名):** 服务器再次告知: “www.baidu.com 的真实名称 (CNAME) 是 www.a.shifen.com。”
- **第二部分 (真实地址):** 然后它提供了真实名称的 AAAA 记录: “www.a.shifen.com 的 IPv6 地址是 2408:873d:22:18cb:0:ff:b037:e6d8和2408:873d:22:1a91:0:ff:b013:cf81。”

6. UDP协议报文格式和交互过程

6.1 实验步骤

1. **产生流量:** 在命令行 (CMD) 中输入 `nslookup www.baidu.com`。如图所示，系统向 DNS 服务器 `172.19.2.2` 发起了查询。
2. **捕获数据:** 启动 Wireshark 捕获流量，并设置过滤器 `dns` 筛选相关数据包。
3. **选取样本:** 在捕获列表选取编号为 **Frame 714** 的数据包作为分析对象。

实验截图:

```
C:\Windows\System32>nslookup www.baidu.com
服务器: UnKnown
Address: 172.19.2.2

非权威应答:
名称: www.a.shifen.com
Addresses: 180.101.51.73
           180.101.49.44
Aliases: www.baidu.com

C:\Windows\System32>
```

No.	Time	Source	Destination	Protocol	Length	Info
713	63.258071	172.19.2.2	10.208.92.63	DNS	295	Standard query response 0x3d33 A ctld1.windowsupdate.com CNAME ctld1.windowsupdate.com.delivery.microsoft.com CNAM...
714	63.259791	10.208.92.63	172.19.2.2	DNS	83	Standard query 0x6a2a AAAA ctld1.windowsupdate.com
715	63.261623	172.19.2.2	10.208.92.63	DNS	279	Standard query response 0x6a2a AAAA ctld1.windowsupdate.com CNAME ctld1.windowsupdate.com.delivery.microsoft.com C...
865	77.356272	10.208.92.63	172.19.2.2	DNS	82	Standard query 0xc6e9 A cloud-cdn.lenovomm.com
866	77.358382	172.19.2.2	10.208.92.63	DNS	258	Standard query response 0xc6e9 A cloud-cdn.lenovomm.com CNAME cloud-cdn.mbgcdn.lenovo.com.cn CNAME cloud-cdn.lenov...
867	77.359861	10.208.92.63	172.19.2.2	DNS	82	Standard query 0x62e1 AAAA cloud-cdn.lenovomm.com
868	77.362134	172.19.2.2	10.208.92.63	DNS	247	Standard query response 0x62e1 AAAA cloud-cdn.lenovomm.com CNAME cloud-cdn.mbgcdn.lenovo.com.cn CNAME cloud-cdn.le...
912	77.761741	10.208.92.63	172.19.2.2	DNS	78	Standard query 0x82fb AAAA osfsr.lenovomm.com
913	77.763610	172.19.2.2	10.208.92.63	DNS	171	Standard query response 0x82fb AAAA osfsr.lenovomm.com CNAME fsr.cn.lenovomm.com SOA ns1.dns5.com
1163	113.095115	10.208.92.63	172.19.2.2	DNS	73	Standard query 0xaa0d AAAA www.baidu.com
1164	113.097067	172.19.2.2	10.208.92.63	DNS	157	Standard query response 0xaa0d AAAA www.baidu.com CNAME www.a.shifen.com SOA ns1.a.shifen.com
1191	114.949310	10.208.92.63	172.19.2.2	DNS	83	Standard query 0x0001 PTR 2.2.19.172.in-addr.arpa
1192	114.951451	172.19.2.2	10.208.92.63	DNS	118	Standard query response 0x0001 No such name PTR 2.2.19.172.in-addr.arpa SOA 19.172.in-addr.arpa
1193	114.981038	10.208.92.63	172.19.2.2	DNS	73	Standard query 0x0002 A www.baidu.com
1194	114.982922	172.19.2.2	10.208.92.63	DNS	548	Standard query response 0x0002 A www.baidu.com CNAME www.a.shifen.com A 180.101.51.73 A 180.101.49.44 NS e.gtld-se...
1195	114.992977	10.208.92.63	172.19.2.2	DNS	73	Standard query 0x0003 AAAA www.baidu.com
1196	114.994689	172.19.2.2	10.208.92.63	DNS	157	Standard query response 0x0003 AAAA www.baidu.com CNAME www.a.shifen.com SOA ns1.a.shifen.com
1210	117.503196	10.208.92.63	172.19.2.2	DNS	76	Standard query 0x3480 A co02.jurassic.net
1211	117.505293	172.19.2.2	10.208.92.63	DNS	549	Standard query response 0x3480 A co02.jurassic.net A 184.233.220.45 NS a.gtld-servers.net NS e.gtld-servers.net NS ...
1212	117.506641	10.208.92.63	172.19.2.2	DNS	76	Standard query 0x20a8 AAAA co02.jurassic.net
1213	117.509273	172.19.2.2	10.208.92.63	DNS	139	Standard query response 0x20a8 AAAA co02.jurassic.net SOA carol.ns.cloudflare.com

Frame 714: 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF... (00:00:00:00:00:01) f0 d4 15 6e 6b bc 08 00 45 00 ... nk ... E

Ethernet II, Src: Intel_E6:6b:bc (f0:d4:15:6e:6b:bc), Dst: IETF_VRRP-VRID_01 (00:00:5e:00:01:01) 00 45 81 08 00 00 00 11 00 00 0a d0 5c 3f ac 13 ... E ... \? ...

Internet Protocol Version 4, Src: 10.208.92.63, Dst: 172.19.2.2 02 02 fe dd 00 35 00 31 15 67 6a 2a 01 00 00 01 ... 5 1 gj* ...

User Datagram Protocol, Src Port: 65245, Dst Port: 53 00 00 00 00 00 00 05 63 74 6c 64 6c 0d 77 69 6e ... c tld1 win

Domain Name System (query) 00 64 6f 77 73 75 70 64 61 74 65 03 63 6f 6d 00 00 ... dowsupda te com ...

1c 00 01


```

    ▶ Frame 714: 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF_{0CCFE3EB-E13A-4FBD-8D49-61FF57B80A0A}, id 0
    ▶ Ethernet II, Src: Intel_6e:6b:bc (f0:d4:15:6e:6b:bc), Dst: IETF-VRRP-VRID_01 (00:00:5e:00:01:01)
    ▶ Internet Protocol Version 4, Src: 10.208.92.63, Dst: 172.19.2.2
    ▶ User Datagram Protocol, Src Port: 65245, Dst Port: 53
      Source Port: 65245
      Destination Port: 53
      Length: 49
      Checksum: 0x1567 [unverified]
      [Checksum Status: Unverified]
      [Stream index: 2]
      [Stream Packet Number: 21]
    ▶ [Timestamps]
      UDP payload (41 bytes)
    ▶ Domain Name System (query)

```

6.2 UDP 报文格式分析

根据实验截图（Frame 714），我们对 UDP 头部进行了详细分析。UDP 头部结构非常简洁，固定长度为 **8 字节**。

UDP 头部字段详细数据表：

字段名称 (Field)	长度	实验捕获 数值	含义与分析
Source Port (源端口)	2 字节	65245	客户端（本机 10.208.92.63）随机开启的高位临时端口，用于接收服务器的回包。
Destination Port (目的端口)	2 字节	53	知名端口，标识接收方（服务器 172.19.2.2）运行的是 DNS 服务。
Length (长度)	2 字节	49	表示 UDP 头部(8字节) + 数据载荷(Payload)的总长度。 计算可知 Payload = 49 - 8 = 41 字节。
Checksum (校验和)	2 字节	0x1567	用于检测数据在传输中是否出错。状态显示 [unverified] 是因为网卡启用了硬件卸载功能，属正常现象。

实验观察：
从 Wireshark 的 "Packet Details" 面板可以看出，UDP 协议直接承载在 IPv4 协议之上。相比于 TCP 协议，UDP 没有序列号、确认号、窗口大小等复杂控制字段，这验证了 UDP **低开销 (Low Overhead)** 的特点。

6.3 UDP 交互过程分析

通过观察 "Packet List" 面板，我们分析 UDP 的交互流程。

1. 请求与响应时序：

- 请求包 (Request):
 - Frame No.: 714
 - 时间: 63.259791
 - 方向: 10.208.92.63 (本机) → 172.19.2.2 (DNS服务器)
 - 内容: Standard query 0x6a2a ...
- 响应包 (Response):
 - Frame No.: 715
 - 时间: 63.261623
 - 方向: 172.19.2.2 (DNS服务器) → 10.208.92.63 (本机)
 - 内容: Standard query response 0x6a2a ...

2. 交互特征分析:

- **无连接建立:** 观察 Frame 714 发送之前, 本机与服务器之间没有进行类似 TCP 的 "三次握手" (SYN 包), 而是直接发送了 Query 数据。
- **请求响应匹配:** 由于 UDP 头部没有序列号, 应用层依靠 **Transaction ID** 进行匹配。
 - Frame 714 (请求) 的 Transaction ID 为 **0x6a2a**。
 - Frame 715 (响应) 的 Transaction ID 同为 **0x6a2a**。
 - 这证明了 Frame 715 是针对 Frame 714 的回复。
- **响应速度:** 从发出请求到收到响应 (63.261623 - 63.259791), 耗时仅约 **1.8ms**, 体现了 UDP 通信的高效性。

实验心得体会

1. 实验中遇到的问题及解决方法

在本次利用 Wireshark 进行端到端网络协议分析的过程中, 我遇到了一些实际操作上的困难, 但通过查找资料和反复尝试, 最终找到了解决办法:

- **问题一: 高流量环境下难以定位目标报文。**
 - **现象描述:** 实验开始时, 由于后台程序和系统服务 (如微信、Windows Update) 产生大量干扰流量, 导致短短几秒内就捕获了数千个数据包, 手动查找特定的 DNS 查询或 TCP 握手包极其困难。
 - **解决方法:** 我熟练掌握了 Wireshark **显示过滤器** 的使用。
 - 对于 DNS 分析, 使用 `dns.qry.name == "www.baidu.com"` 精确筛选。
 - 对于 TCP 分析, 利用右键菜单中的 **"追踪流 (Follow TCP Stream)"** 功能, 系统自动生成 `tcp.stream eq X` 过滤器, 成功屏蔽了背景噪声, 只保留了关注的那一条完整会话。
- **问题二: 抓包时无法捕获到完整的 HTTP 请求。**
 - **现象描述:** 在浏览器输入网址回车后, Wireshark 中只有 TCP 握手, 却没有捕捉到 `GET` 请求, 或者 DNS 查询没有发生。
 - **解决方法:** 分析认为是浏览器缓存和操作系统 DNS 缓存导致请求未真正发出。我使用了 `Ctrl+Shift+Delete` 清空浏览器缓存, 并在命令行执行了 `ipconfig /flushdns` 命令, 强制主机重新发起完整的解析和请求过程, 随后成功捕获。

2. 实验中产生的错误及原因分析

在分析报文细节时, 我观察到了一些看似 "错误" 的数据现象, 经过深入分析, 纠正了我的认知误区:

- **现象: UDP/TCP 校验和 (Checksum) 显示为 "Unverified" 或错误。**
 - **错误认知:** 起初看到 Wireshark 在 UDP 头部 (Frame 714) 提示 `Checksum: 0x1567 [unverified]` 甚至标红时, 我误以为是实验环境网络质量差导致数据传输出现了误码。
 - **原因分析:** 经过查阅资料, 我了解到这是由于现代网卡的 **Checksum Offload (校验和卸载)** 机制。为了降低 CPU 负荷, 操作系统将计算校验和的工作交给网卡硬件完成。Wireshark 捕获的是 CPU 发给网卡之前的原始数据, 此时校验和尚未计算 (通常为 0 或随机值), 因此软件认为校验和错误。这属于正常现象, 并非实验操作失误。
- **现象: DNS 解析结果与预期不符 (CNAME 链)。**

- **错误认知**: 在使用 `nslookup` 时, 我预期直接得到 `www.baidu.com` 的 IP 地址, 但数据包中却先出现了 `www.a.shifen.com`。
- **原因分析**: 这反映了我对 DNS 别名机制理解不够深入。大型网站为了 CDN 负载均衡, 使用了 CNAME 记录。Wireshark 的 Answer 字段清晰地展示了从 `User Domain` -> `CNAME` -> `A/AAAA Record` 的完整解析链条, 纠正了我“一个域名直接对应一个 IP”的简单线性思维。

3. 体会与收获

本次实验将枯燥的网络协议理论转化为了屏幕上真实跳动的字节, 让我受益匪浅:

1. 理论与实践的深度融合:

以前学习 TCP“三次握手”和“四次挥手”时只能死记硬背状态图, 而现在通过 Wireshark 的时序图, 我亲眼验证了 SYN、ACK、FIN 标志位的翻转以及序列号的递增逻辑。这种可视化的学习过程, 让我对“可靠传输”的实现机制有了具象化的理解。

2. 对协议特性的直观感悟:

通过对比 TCP 和 UDP 的报文结构, 我深刻体会到了设计的权衡 (Trade-off) :

- **TCP** 头部复杂 (20+字节), 握手过程繁琐, 但换来了数据传输的可靠性。
- **UDP** 头部极简 (8字节), 无连接即发即走, 完美契合了 DNS 这种对速度要求高、对丢包可容忍的应用场景。

3. 工程素养与安全意识的提升:

在 HTTP 分析环节, 看到文件名和 User-Agent 以**明文**形式呈现, 让我受到极大的触动, 深刻意识到 HTTP 协议的不安全性以及 HTTPS 加密的必要性。同时, 掌握 Wireshark 这一工业级工具的使用, 不仅提升了我的动手能力, 更培养了我用“数据驱动”的思维去排查网络故障的工程素养。